

1. Introduction & Context of Online Evaluation

- **Transitioning from Offline Evaluation to Online Production Monitoring:** Today marks our final session dedicated to **RAG (Retrieval Augmented Generation) evaluations**, where we focus on **Online Evaluation**. In our prior sessions, we built a comprehensive **offline evaluation suite** covering approximately 14 to 15 distinct metrics. We completed our offline evaluation pipeline by tying these metrics together using **regression testing** to verify whether modifying a system component causes performance regression prior to deployment. Completing offline evaluation allows us to thoroughly test application quality before pushing the software live into production. Once testing completes, we deploy the system into production where end users begin actively interacting with the live application. However, deployment does not mean evaluation stops, ideal organizations continuously evaluate LLM applications and chatbots post-deployment in a process called **Online Evaluation**.
 - **Core Pillars of Online Infrastructure:** Online evaluation encompasses **Observability**, **Monitoring**, and **Evaluation** within live production settings using specialized tooling such as **LangSmith** and **LangFuse**.
 - **Operational Baseline:** For our practical setup today, we assume that our software application the **CampusX Doubt Solver** chatbot is deployed and running live in production. Our plan of action involves continuously running evaluation tests within this live production environment, capturing incoming production data, and observing system health to detect any system breakdowns.
-

2. Theoretical Breakdown: Offline vs. Online Metric Compatibility

- **Audit of Offline Metric Suite:** To construct our online evaluation suite, we first audit the 14 to 15 metrics previously built for offline evaluation across five distinct architectural layers:
 1. **Retriever Metrics:** **Contextual Recall** and **Contextual Precision**.
 2. **Generator Metrics:** **Faithfulness** and **Answer Relevancy**.
 3. **RAG Pipeline Metrics:** **Contextual Relevancy**, **Faithfulness**, and **Answer Relevancy**.
 4. **Application Level Metrics:** **Correctness**, **Completeness**, and **Style Check**.
 5. **Safety Metrics:** **Scope Leakage** and **Toxicity**.
 6. **Operational Metrics:** **Latency**, **Cost**, and **Reliability**.
- **The Hard Truth of Online Metric Execution:** The hard truth of online evaluation is that we cannot run all 14 to 15 offline metrics in a live production environment.
- **The Reference Answer Bottleneck:** The fundamental reason for this limitation is the absence of a **reference answer** or **ground truth answer** in live settings. In **Offline Evaluation**, we

construct **Golden Datasets** where every question is paired with a verified reference answer, enabling the evaluation of **reference based metrics** like **Correctness**. In **Online Evaluation**, live users submit completely novel questions in real time, meaning we do not possess predefined ground truth answers for future user inputs. Without reference answers, any **reference-based metric** cannot be computed online.

- **Reference Dependent vs. Reference Free Categorization:**

1. **Contextual Recall:** Requires reference information specifying what the ideal retrieved chunk looks like, making it reference dependent and impossible to evaluate online.
2. **Contextual Precision:** Requires reference answers to evaluate retrieval ordering, making it reference dependent and impossible to evaluate online.
3. **Correctness:** Requires comparing generated output against ground truth answers, making it reference dependent and impossible to evaluate online.
4. **Completeness:** Requires reference answers to verify whether all required key points were included, making it reference dependent and impossible to evaluate online.
5. **Faithfulness:** Measures whether generated answers are grounded in retrieved context without needing ground truth answers, making it a **reference free** metric that can be evaluated online.
6. **Answer Relevancy:** Evaluates how well the response addresses the query without needing ground truth answers, making it a **reference-free** metric that can be evaluated online.
7. **Contextual Relevancy:** Evaluates whether retrieved chunks are relevant to the user query without ground truth answers, making it a **reference free** metric that can be evaluated online.
8. **Style Check:** Evaluates stylistic properties without ground truth answers, making it a **reference free** metric that can theoretically be evaluated online.
9. **Safety Metrics (Scope Leakage, Toxicity):** Analyze input and output content independently of ground truth answers, making them **reference free** metrics that can be evaluated online.
10. **Operational Metrics (Latency, Cost, Reliability):** Log system performance metrics directly without requiring reference answers, allowing them to be evaluated online.

- **Scoping the Production Online Evaluation Suite:**

1. We skip separate component-level generator evaluations and focus on the **RAG Triad (Contextual Relevancy, Faithfulness, Answer Relevancy)** to assess overall quality.
2. We exclude **Style Check** because style scores exhibit low variance between offline and online settings—such as an offline score of 80 rarely dropping to 30 online—and to respect session time limits.
3. We select **Toxicity** as our primary safety metric to demonstrate the pattern while

keeping execution efficient.

4. We select **Latency** and **Cost** for operational metrics, omitting **Reliability** because it requires external monitoring infrastructure like **Grafana** and **Prometheus**, which fall beyond our current scope.
 5. Thus, our final **Online Evaluation Suite** tracks **6 core metrics: Contextual Relevancy, Faithfulness, Answer Relevancy, Toxicity, Latency, and Cost**.
-

3. Operational Metrics Setup: Latency & Cost Logging

- **Simplicity of Operational Telemetry:** We begin implementation with **Operational Metrics (Latency and Cost)** because they are the easiest to configure in **LangSmith**. Simply connecting our application code to LangSmith automatically triggers telemetry logging for latency and token costs without requiring manual evaluation functions.
- **Connecting Application Code to LangSmith:**
 1. Access `smith.langchain.com` and navigate to the Tracing section.
 2. Create a new application project named `cx-doubt-solver` to prevent naming conflicts.
 3. Generate a LangSmith API key and update the local `.env` configuration file with environment variables: `LANGSMITH_TRACING=true`, `LANGSMITH_ENDPOINT`, `LANGSMITH_API_KEY`, and `LANGSMITH_PROJECT=cx-doubt-solver`.
- **Initial Trace Logging & Asynchronous Architecture:** Executing `rag_pipeline.py` sends query execution telemetry to LangSmith, capturing initial execution time and monetary cost. **Asynchronous Tracing** ensures LangSmith performs tracing asynchronously behind the scenes, so telemetry collection does not add additional latency to application execution.
- **Identifying the Implicit Tracing Defect:**
 1. Examining initial trace visualizer graphs reveals that implicit tracing only captures **Prompt Template** creation, **ChatOpenAI** (LLM generation), and **String Output Parser** steps.
 2. It completely misses the **Retrieval** step (fetching document chunks from the vector database) and the **Re-ranking** step.
 3. Because retrieval and re-ranking are omitted, the initial reported latency of 4.14 seconds underestimates actual end-to-end pipeline execution time.
- **Resolving Tracing Gaps with @traceable Decorators:**
 1. We install the `langsmith` package via `uv add langsmith`.
 2. We explicitly decorate the core entry point `invoke()` in `rag_pipeline.py` with `@traceable(name="rag_pipeline")`.

3. We decorate the `invoke()` function in the re-ranker file with `@traceable(name="retriever")`.
 4. Decorators establish a proper parent-child hierarchy in telemetry logs without duplicating traces.
 5. Re-running the pipeline captures the full execution tree including retrieval, re-ranking, prompt formatting, LLM generation, and parsing—yielding accurate end-to-end latency of 4.79 seconds and total token cost.
- **Simulating Production Traffic:** To generate telemetry data for monitoring dashboards, we execute our pipeline against a series of diverse domain queries. Queries include: What is GSM8K?, What is MMLU?, Why do we need LLM evals?, What are LLM evals?, What are online evals?, What is the difference between model evals and application evals?, and What are benchmarks?. This manual execution replicates live user interaction, accumulating 8 to 9 production traces in LangSmith.
-

4. Production Observability Dashboard & Alerting System

- **Custom Dashboard Creation:** Navigate to the **Monitoring** tab in LangSmith. Rather than relying on default pre-built dashboards, we create a **Custom Dashboard** named `cx-doubt-solver-dashboard` to tailor visualization.
- **P95 Latency Tracking:**
 - Configure a chart tracking **Latency** using **Percentile 95 (P95)** over specified time windows, such as the Last 1 Hour.
 - **P95 Latency** measures tail latency, reflecting the maximum response time experienced by 95% of users and highlighting performance for dissatisfied users facing delays.
- **Average Cost Tracking:**
 - Configure a chart tracking **Total Cost** aggregated by **Average** per query run across the time window.
 - Tracking average cost per query provides insight into unit economics per user interaction rather than total cumulative spending.
- **Configuring Automated Threshold Alerts:**
 - Navigate to the **Alerts** tab to set up automated notifications via Slack when operational thresholds are breached.
 - Example logic: Raise an alert if total LLM cost exceeds \$5 over the last 60 minutes.
 - **Technical Limitation in LangSmith Alerting:** LangSmith restricts latency alerts to **Average Run Execution Time** rather than P95 percentile latency.

- Reasoning behind limitation: P95 percentile tracks statistical outliers; triggering alerts on individual outlier spikes could produce false alarms, whereas average latency reflects performance degradation impacting the broader user population.
 - **Alignment with Service Level Objectives (SLOs):** Operational alert thresholds must not be set arbitrarily by engineers; they must derive directly from business-defined **Service Level Objectives (SLOs)**. Different business use cases dictate different acceptable latency boundaries, such as complex research synthesis agents naturally requiring higher latency limits than simple Q&A bots.
-

5. Safety Metrics Implementation: Online Toxicity Evaluation

- **Architectural Options for Online Safety:**
 - Option 1: Re-implement custom offline safety evaluators in online pipelines.
 - Option 2: Utilize built in, reference free evaluators provided directly by LangSmith, such as PII Leakage, Prompt Injection, Code Injection, Toxicity, Bias & Fairness.
- **Selecting LangSmith's Built-in Toxicity Evaluator:** We select LangSmith's built in **Toxicity Evaluator** because safety evaluation prioritizes reliable toxicity detection over custom implementation, leveraging optimized evaluator prompts built by specialized researchers.
- **Evaluator Configuration:**
 - Select LLM-as-a-judge model provider, such as GPT-4o or GPT-4.5.
 - Maintain the default system prompt optimized for toxicity scoring.
 - Set target source to **Tracing** for online live runs rather than **Datasets** for offline evaluation.
 - Provide an OpenAI API key to cover evaluation inference costs.
- **Sampling Rate Optimization & Cost Management:**
 - Setting **Sampling Rate** to 100% evaluates every production trace, which significantly increases LLM judge API expenses in high-volume environments.
 - In production, we configure sampling rates, such as 30%, to evaluate a statistically representative sample of user traffic, estimating population toxicity while staying within budget.
 - For testing and demonstration, we temporarily set the sampling rate to 100%.
- **Asynchronous Execution & Visualization:**
 - Online evaluators run asynchronously in the background, adding feedback scores ranging from 0 to 100 and detailed natural language explanations after a 2 to 3 minute delay.
 - We integrate the Toxicity feedback metric into our custom dashboard to track average

toxicity over time.

6. Quality Metrics Implementation: Online RAG Triad Evaluation

- **The Golden Rule of Quality Evaluation Consistency:**
 - Offline and online evaluations for quality metrics **Contextual Relevancy**, **Faithfulness**, and **Answer Relevancy** must use the exact same evaluation framework, such as **DeepEval**.
 - Using different evaluators, such as DeepEval offline versus LangSmith built-in evaluators online, introduces non-comparable baselines due to differing prompt structures and internal scoring algorithms, such as a score of 45 in DeepEval versus 55 in LangSmith.
 - Evaluating with identical frameworks ensures numerical changes, such as a drop from 90 offline to 85 online, reflect genuine application degradation rather than evaluation artifact variance.
 - **Production Architecture for Online Quality Evaluation:**
 - Host a background service or **cron job** on the production server, such as AWS, configured to trigger periodically every 5 minutes.
 - The cron job executes an evaluation script named `eval_online.py`.
 - `eval_online.py` uses the **LangSmith SDK Client** to pull recent production traces logged over the previous 5-minute window.
 - The script passes trace inputs, retrieved contexts, and generated outputs into our offline **DeepEval** evaluator objects powered by GPT-4o-mini.
 - DeepEval computes metric scores and `eval_online.py` posts feedback scores back to LangSmith via SDK to associate scores with specific traces.
 - **Local Simulation & Live Logging:**
 - We simulate the cron service locally using an infinite `while` loop with a 60-second sleep interval executing `eval_online.py`.
 - Executing new production queries triggers the SDK background script, which evaluates traces and populates feedback scores for **Answer Relevancy**, **Contextual Relevancy**, and **Faithfulness** in LangSmith.
 - We add individual quality metric charts to our custom dashboard to observe real-time quality fluctuations across production traffic.
-

7. Closing the Data Flywheel: Production Failure Harvesting & Dataset Versioning

- **Linking Online Failure Modes to Offline Testing:** Offline evaluation datasets must continuously evolve by incorporating real world production failures. When production monitoring detects low scores on quality or safety metrics for a trace, that trace represents a valuable edge case for system improvement.
- **Human-in-the-Loop Dataset Augmentation:**
 - In LangSmith, engineers can select failed production traces and click **Add to Dataset**.
 - A domain expert reviews the failed query and system output, writes a verified reference answer, and saves the pair into a **Golden Dataset**.
 - This reference-augmented record expands offline evaluation coverage, preventing future regressions on identical edge cases.
- **Centralized Dataset Management & Version Control:**
 - Managing Golden Datasets directly inside LangSmith provides a single source of truth accessible by both offline evaluations and online dataset curation workflows via SDK.
 - LangSmith automatically tracks **dataset versioning**, allowing teams to inspect historical dataset states, such as rolling back to dataset versions from 6 months prior, and audit dataset growth over time.